

Last modified on

Sports Data Soccer API - Match Provisional LineUps Feed (MA46)

Get provisional lineup information prior to the game.

Overview

The Soccer **Provisional LineUps feed** is designed to provide information around provisional squads that may participate in the fixture.

It details the following information:

- Key match info (players and team)
- Line Up Information
- Detailed Player Information

Required Compliance:

To comply with our API security and best practice logic for your API calls please refer to our **FAQs**. We also recommend that you take a look at our **API Overview and Authentication Guide**. You will learn the basics of how our API works and key concepts to get started.

How Often Should I Poll The Feed?

We have provided some guidelines to help ensure that you call the feed only as often as you need to. Make sure that you don't exceed the rate confirmed with us - this can vary depending on whether you make requests as a B2B or B2C outlet - if you need more information on this, contact your Stats Perform representative.

UPDATED	POLLING FREQUENCY
Once in 24 hours, available 4 days ahead of the scheduled fixture	

Match Provisional LineUps (MA46) Guide

USAGE
GET Match Provisional Lineups information for a fixture by passing the fixture/match UUID as a path parameter (the asset/resource) in
GET Match Provisional Lineups information for a fixture by passing a fixture UUID to the fx parameter.

USAGE

GET Match Provisional Lineups information for a **fixture** by passing legacy Opta/OptaCore fixture ID

Query Parameters And Example Request Parameters

This feed also makes use of the **Global query parameters**, and you can combine them with the following query parameters (some of which can be used in conjunction with each other). However, you can query the

`/matchprovisionallineups` feed by specific fixture (match) UUID/tournament calendar UUID.

Legacy IDs: You can use the following query parameter.

- `fx` - only for a single match.

Important: If you call the feed using multiple filters by ID then only one type of ID can be passed to all parameters - this means choosing to use either only the 'UUID' type, or all 'oc id' type, or all 'op id' type (for example, all 'UUID' type: `comp=uuid&stg=uuid`). You cannot mix ID types (for example, `comp=uuid&stg=urn:perform:optacore:stage:ocid`); otherwise, the feed will return an error.

How to use query parameters and make successful requests:

In our guides, we use HTTPS syntax for our example GET requests (rather than cURL, for example). All URLs are case-sensitive - this means that URL strings must contain the correct case and operators. You **MUST** format your requests correctly and include certain information, otherwise, an error will be returned. You can find information about the API, including the base URL (`{protocol}://{requestDomain}/{feedResource}`), in the **main guide** and **API Overview and Authorisation guide**.

- Include the following information in the URL or header and make sure it is correct for your outlet: your unique and valid `{outletAuthKey}` (Outlet Authentication Key), query parameters for `_rt={mode}` (operating mode) and `_fmt={dataFormat}` (response format). If required, it should also include an entity UUID (such as a fixture/match UUID).
- Begin the query parameter string with `?` (question mark) - this must be after the Outlet Authentication Key (and endpoint, if applicable). To build up a query string, make sure that each query parameter is separated by the `&` (ampersand) operator. Values must be separated by the correct operator, for example `&` (multiple values, 'AND'), or - if a parameter supports it - `,` (multiple values, 'OR') or `!` ('NOT').
- Use the required `_` (underscore) for any parameters listed with this prefix.
- Remove any placeholder value curly braces `{ }` from your calls (we include these as an example only).
- If using the JSONP format (`_fmt=jsonp`) you **MUST** use it in combination with the callback parameter (`_clbk`) and a fixed value (or where you use a different value in a different instance, you must do this as little as possible). Be aware that common JavaScript frameworks, such as jQuery, can default to use randomly generated function names. You must make sure that these are overridden and define a fixed function name instead.

Query parameter	Value and description
fx	GET Match Provisional Lineups information for a fixture by passing the fixture/match UUID as a path parameter (the asset/resource) in the URL. GET https://api.performfeeds.com/soccerdata/matchprovisionallineups/{outletAuthKey}/fx={fixtureUuid}?_fmt={dataFormat}&_rt={mode};
fixture/match UUID	GET Match Provisional Lineups information for a fixture by passing legacy Opta/OptaCore fixture ID. GET https://api.performfeeds.com/soccerdata/matchprovisionallineups/{outletAuthKey}{fx=urn:perform:opta:fixture:{optaId}}&_fmt={dataFormat}&_rt={mode};

Sample Calls

These sample calls let you see an example URL and the response - click on the links below (hyperlinks open in a new window).

- XML
- JSON

Feed Output

Here, you can find an explanation of all possible response fields and data. This is XML but fields in the feed response are similar for JSON.

- ▶<matchProvisionalLineUps> (XML only)
- ▶<matchInfo>
- ▶<description>
- ▶<sport>
- ▶<ruleset>
- ▶<competition>
- ▶<country>
- ▶<tournamentCalendar>
- ▶<stage>
- ▶<series>
- ▶<contestants> (XML only)

▶<contestant>

▶<country>

▶<venue>

▶<provisionalLineUps>

▶<lineUp>

▶<player>

▶<teamOfficial>

▶<injuries>

▶<injury>

▶<other>

▶<suspensions>

▶<suspension>